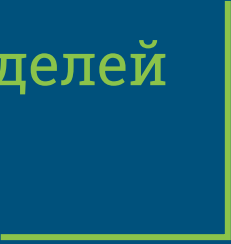




Современные архитектуры ИС



Лекция 7
Масштабирование языковых моделей



План лекции

1. **Обучаемое разреженное внимание**
2. Сжатое внимание
3. Self-supervised learning
4. Мультимодальность
5. Законы масштабирования

Обучаемое разреженное внимание

Шаблоны связей в шаблонном разреженном внимании задаются независимо от входных данных.

Однако, разные токены могут требовать разного набора связей.

Поэтому логично будет модели самостоятельно отбирать, какие связи оставлять, а какие — отбрасывать.

Lightning Indexer

Такой индексатор дает вычисляет оценку между текущим запросом и предыдущими токенами

$$I^{t,s} = \sum_{j=1}^{n_h} w_j^t \text{ReLU}(q_j^t k_j^s)$$

По сути мы перед вычислением механизма внимания делаем его приближенную оценку.

Первая стадия обучения индексатора

Индексатор необходимо обучить, чтобы он работал корректно.

На первой стадии заморозим слой внимания и будем обновлять параметры только параметры индексатора. Здесь внимание остаётся плотным, индексатор выучивает распределение весов плотного внимания.

$$L^I = \sum_t D_{KL} \left\langle p^{t,:} \parallel \text{Softmax}(I^{t,:}) \right\rangle$$

Вторая стадия обучения индексатора

Во второй стадии обучения веса внимания размораживаются. Теперь для каждого токена индексатор выбирает множество из k самых важных позиций

$$s^t = \text{Top}_k(I^{t,\cdot})$$

Распределение выходов внимания меняется. Поэтому распределение индексатора тоже необходимо обновлять

$$L^I = \sum_t D_{KL} \left\langle p^{t,s^t} \parallel \text{Softmax}(I^{t,s^t}) \right\rangle$$

Эффективность DSA

Такой индексатор требует дополнительных вычислений, синхронизаций и, возможно, даже коммуникаций при обучении на длинном контексте

Эффективность такого индексатора напрямую зависит от того, насколько дешево он устроен и насколько эффективно реализованные его вычисления на GPU.

Например, мы можем использовать вычисления в более низкой точности по сравнению с механизмом внимания, например, fp8 или даже fp4.

План лекции

1. Обучаемое разреженное внимание
2. **Сжатое внимание**
3. Self-supervised learning
4. Мультимодальность
5. Законы масштабирования

Скалярные коэффициенты

Попробуем сжимать информацию, поступающую на вход механизму внимания. Будем развивать идеи MLA и спроецируем входы в пространство меньшей размерности

$$C = HW^{KV},$$

$$C \in \mathbb{R}^{n \times d_c}$$

Мы сжали матрицу H по оси d, теперь сожмем по оси n. Объединим 4 токена в один вектор.

$$c_1^{\text{comp}} = 0.25c_1 + 0.25c_2 + 0.25c_3 + 0.25c_4$$

Векторные коэффициенты

Простые скалярные веса не будут зависеть от входных данных. Сделаем их векторными и зададим следующим образом:

$$S^{\top} = \text{Softmax} \left[(HW^Z)^{\top} \right], \quad S \in \mathbb{R}^{n \times d_c},$$

Получаем:

$$c_1^{\text{comp}} = \sum_{i=1}^4 s_i \odot c_i, \quad c_2^{\text{comp}} = \sum_{i=5}^8 s_i \odot c_i, \quad \dots,$$

Пересекающиеся группы

Сейчас мы создаем искусственные границы между группами токенов, поэтому сделаем их пересекающимися группами

$$c_1^{\text{comp}} = \sum_{i=1}^4 s_i \odot c_i, \quad c_2^{\text{comp}} = \sum_{i=1}^8 s_i \odot c_i, \quad c_2^{\text{comp}} = \sum_{i=5}^1 2s_i \odot c_i, \quad \dots$$

Разделим матрицы W^{KV} и W^{Z} на матрицы для текущей группы и для прошлой и получим

$$c_1^{\text{comp}} = \sum_{i=1}^4 s_{\text{curr},i} \odot c_{\text{curr},i}, \quad c_2^{\text{comp}} = \sum_{i=5}^8 s_{\text{curr},i} \odot c_{\text{curr},i} + \sum_{i=1}^4 s_{\text{prev},i} \odot c_{\text{prev},i}, \quad \dots$$

Немного про вычисления на GPU

Но почему мы вычисляем так?

$$S_{\text{curr}}^{\top} = \text{Softmax}\left[(HW_{\text{curr}}^Z)^{\top}\right],$$

а не

$$S_{\text{curr}}^{\top} = \text{Softmax}\left[(C_{\text{curr}}W_{\text{curr}}^Z)^{\top}\right],$$

В первом случае, вычисления пар матриц S и C можно сделать параллельно в одном кернеле. Во втором — придется вычислять их последовательно

Сжатие в механизме внимания

Применим механизм сжатия в механизме внимания. Сожмем входы и подадим результат на все K и V головы. Это позволит нам увеличить размерности голов K и V .

Низкоранговая матрица W^Q

Матрица W^Q в таком случае становится огромной. Поэтому воспользуемся низкоранговым подходом

$$Q = HW_{\text{down}}^Q W_{\text{up}}^Q$$

Группировка выходных проекций

Аналогичная проблема возникает и с выходной матрицей O . Разделим ее на группы и спроецируем с помощью матрицы W_{down}

$$O_i \in \mathbb{R}^{n \times \left(\frac{c \times h}{g}\right)} \quad W_{\text{down}}^O \in \mathbb{R}^{\left(\frac{c \times h}{g}\right) \times n_g} \quad O_i^G = O_i W_{\text{down}}^O, \quad O_i^G \in \mathbb{R}^{n \times n_g}$$

После этого различные группы конкатенируем в одну матрицу и проецируем в исходное пространство с помощью W_{up}

Такой подход называется **группировкой выходных проекций**

CSA, HCA

Объединение механизма сжатия токенов, DSA и группировки выходных проекций с механизмом внимания дает механизм сжатого разреженного внимания CSA.

CSA имеет коэффициент сжатия 4 к 1. Но DeepSeek также представили механизм сильносжатого внимания HCA со сжатием 128 к 1. При таком сжатии даже нет необходимости использовать разреженное внимание.

План лекции

1. Обучаемое разреженное внимание
2. Сжатое внимание
3. **Self-supervised learning**
4. Мультимодальность
5. Законы масштабирования

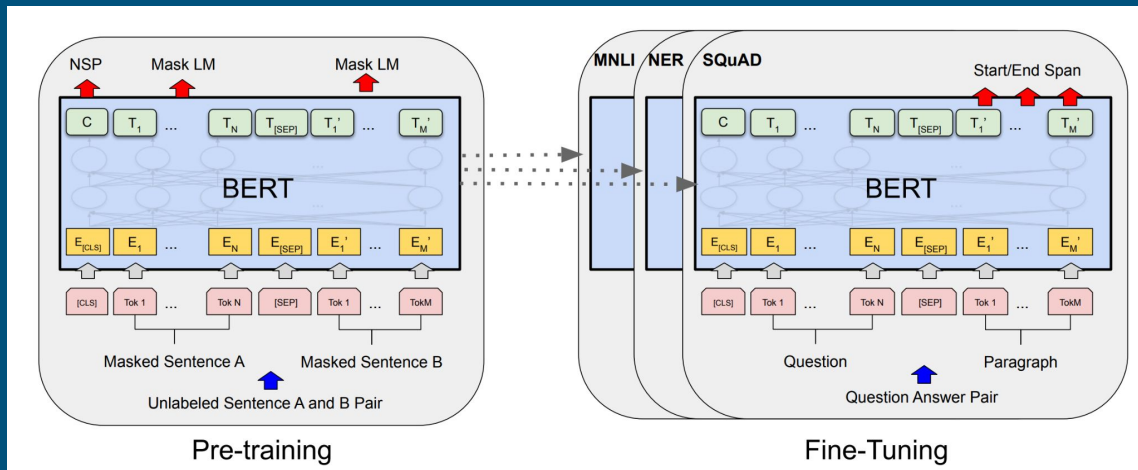
Что такое SSL

Supervised learning — обучение с учителем, то есть по размеченным данным

Unsupervised learning — обучение без учителя, то есть когда размеченных данных нет

Self-supervised learning — обучение с самоконтролем, когда размеченных данных как бы нет, но мы можем получить задачу supervised learning из имеющихся данных на pretext task, чтобы подготовить веса к downstream task.

Архитектура BERT



BERT — трансформер-кодировщик. Часто его используют для того, чтобы сжать предложение в вектор

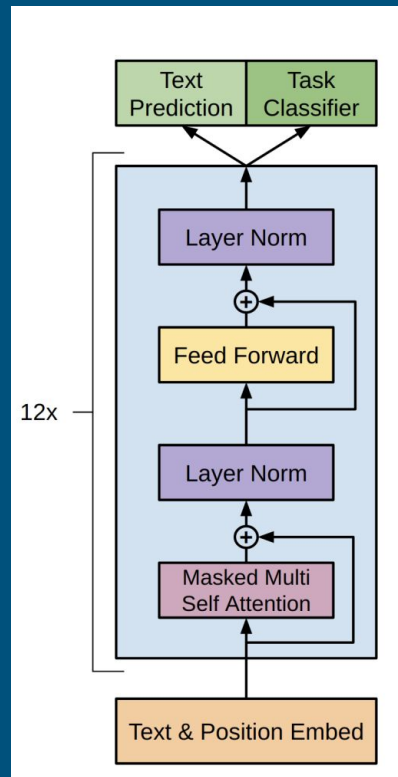
Предобучение BERT

Для предобучения использовали 2 задачи:

- Обучение на маскированной языковой модели
- Является ли данное предложение – следующим предложением для текущего текста

GPT

GPT – трансформер-декодировщик, обучается на задаче предсказания следующего токена.



План лекции

1. Обучаемое разреженное внимание
2. Сжатое внимание
3. Self-supervised learning
4. **Мультимодальность**
5. Законы масштабирования

Что такое мультимодальность?

Мультимодальность в нейронных сетях означает способность модели обрабатывать информацию из разных типов данных — модальностей.

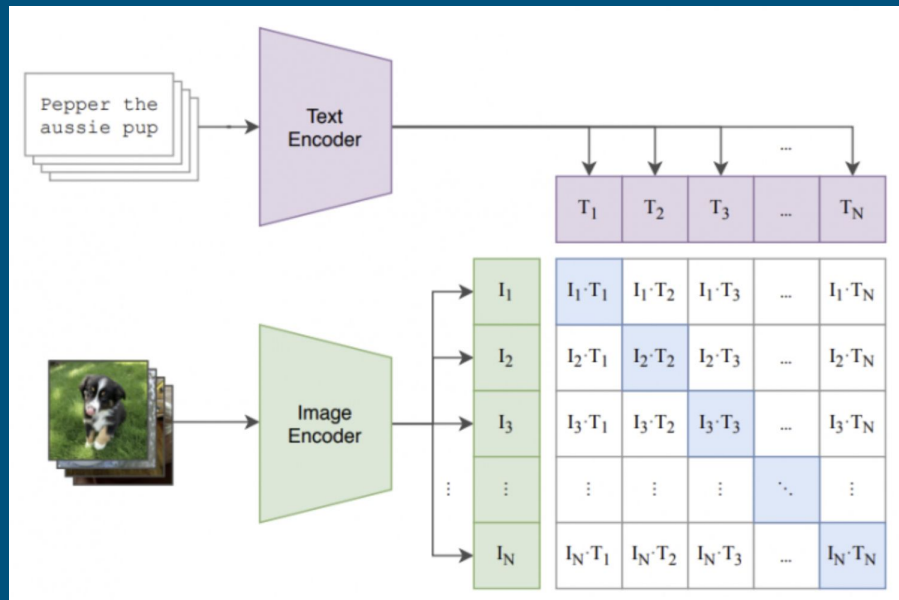
Модальностями могут быть, например, изображения, текст, аудио, видео и тд.

Граница между модальностями часто зыбкая. В этом смысле лучше говорить не о строгой категоризации, а о переходе от более однородных и схожих данных к структурно более разнородным

CLIP

Если мы закодируем изображения с помощью ViT, то его токены будут лежать в одном семантическом пространстве.

Если закодируем текст с помощью BERT — то эти токены будут лежать в другом семантическом пространстве.



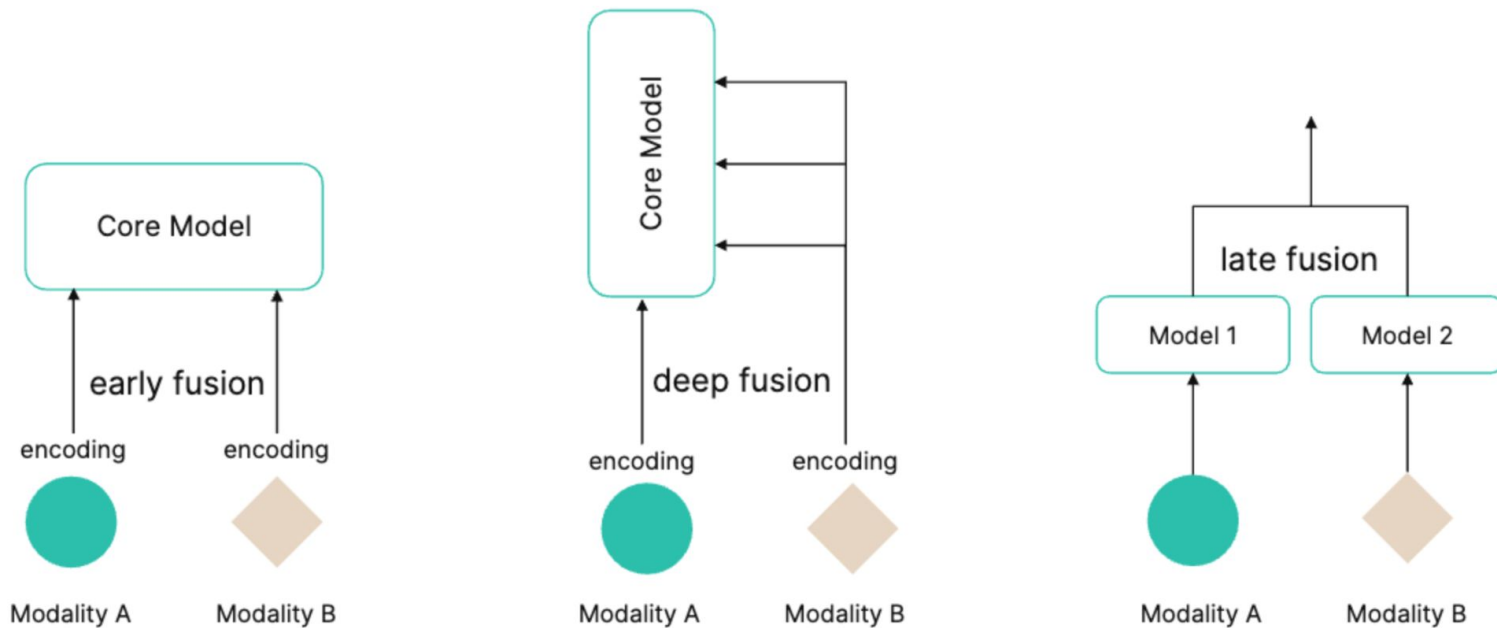
Кросс-модальность трансформеров

Трансформеры способны успешно обрабатывать и текст, и изображения.

Текст — это последовательность, а изображения — двумерная структура. Но трансформер воспринимает все входные данные как неупорядоченные множества

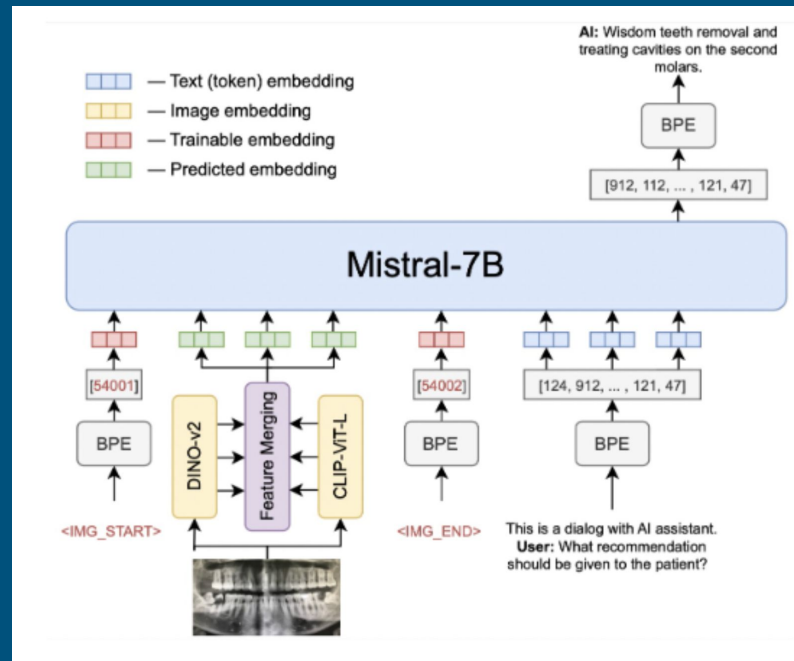
В этом смысле главное поставить задачу так, что эти данные лежали в одном семантическом пространстве

Фьюз модальностей



Merge of Encoders

Разные визуальные кодировщики
могут быть полезны для разных
задач



План лекции

1. Обучаемое разреженное внимание
2. Сжатое внимание
3. Self-supervised learning
4. Мультимодальность
5. **Законы масштабирования**

Законы масштабирования

У нас есть данные (мы получаем их с помощью SSL и например мультимодальности)

У нас есть удачно масштабируемая модель — трансформер (со всеми нашими архитектурными и инженерными улучшениями)

У нас есть вычислительные мощности и способ их организации.

Время масштабироваться до LLM.

Законы масштабирования

Если увеличивать размер модели, зафиксировав количество данных, то лосс будет падать

Если увеличивать количество обучаемых данных, зафиксировав размер модели, то лосс тоже будет падать

Если увеличивать и то и другое, то лосс будет падать.

Законы масштабирования

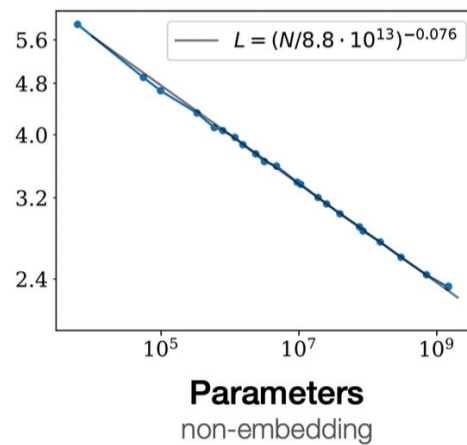
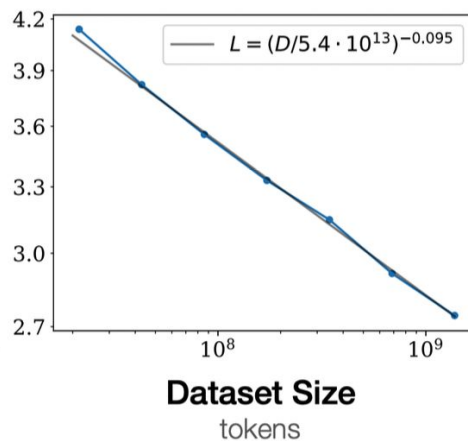
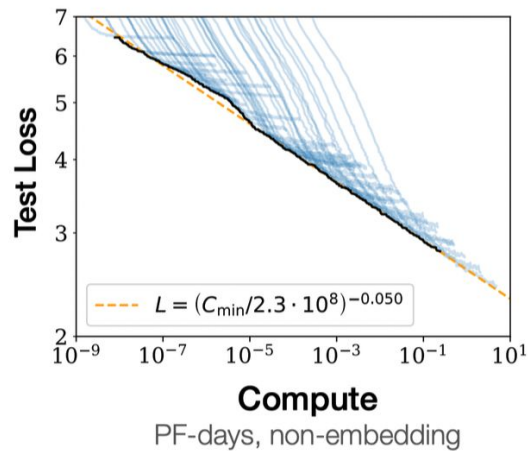
Если увеличивать размер модели, зафиксировав количество данных, то лосс будет падать

Если увеличивать количество обучаемых данных, зафиксировав размер модели, то лосс тоже будет падать

Если увеличивать и то и другое, то лосс будет падать.

Но как конкретно?

Законы масштабирования



$$\mathcal{L} \sim \left(\frac{1}{7.3} C \cdot 10^8 \right)^{-0.050}, \quad \mathcal{L} \sim \left(\frac{1}{5.4} D \cdot 10^{11} \right)^{-0.095}, \quad \mathcal{L} \sim \left(\frac{1}{8.8} N \cdot 10^3 \right)^{-0.027}$$

Совместный закон масштабирования

Эмпирически можно получить совместный закон масштабирования для данных и параметров при фиксированном компьютере:

$$\begin{cases} \mathcal{L}(N, D) = c + AN^{-\alpha} + BD^{-\beta}, \\ C(N, D) = C_0ND \end{cases}$$

Законы масштабирования и bias-variance trade-off

Через совместный закон масштабирования можно продолжить идею, что увеличение количество параметров снижает смещение, а увеличение количества данных — снижает разброс

$$\text{bias} \sim N^{-\alpha}, \quad \text{variance} \sim D^{-\beta}$$

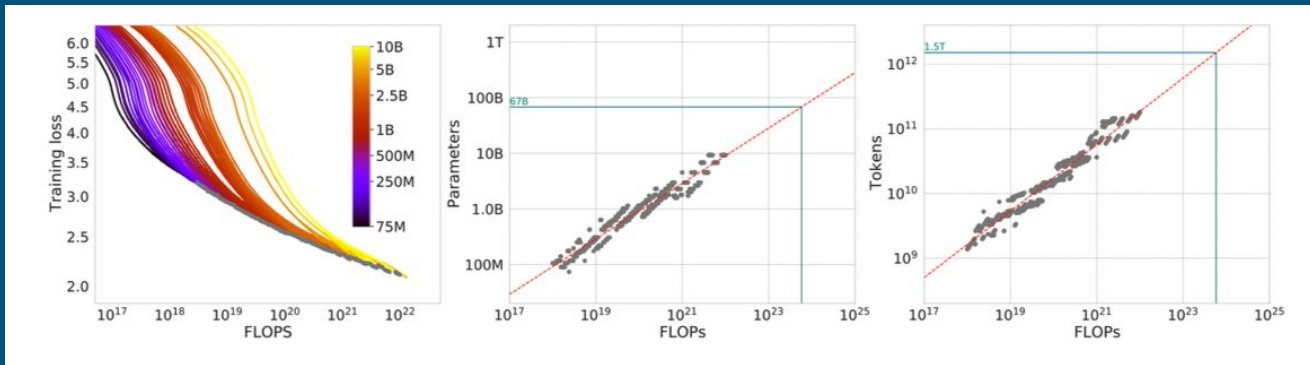
Можно говорить о том, что при недообучении отношение R малое, а при переобучении — высокое

$$R(N, D) = \frac{\text{variance}}{\text{bias}} \sim N^{\alpha} D^{-\beta}$$

Закон Шиншиллы

Чтобы достичь bias-variance компромисса при фиксированном компьютере $C \sim ND$, необходимо найти функцию $D=f(N)$, гарантирующую оптимальное отношение R .

Найдем минимум по объединению кривых лоссов для разных моделей на разных датасетах при фиксированном компьютере



Закон Шиншиллы

Закон Шиншиллы — “оптимальное” соотношение количества обучающих данных к количеству параметров модели

$$D = 20N$$

Закон Шиншиллы

Закон Шиншиллы — “оптимальное” соотношение количества обучающих данных к количеству параметров модели

$$D = 20N$$

Но это не железное правило. “Оптимальное” соотношение будет зависеть от архитектуры модели

Вопросы?
